🔷 1. What is LINQ?

LINQ (Language Integrated Query) is a feature in C# that allows you to query collections (like arrays, lists, XML, databases) using a **SQL-like syntax directly inside C#**.

👉 Before LINQ:

- More loops (foreach)
- More conditional logic
- Less readable

👉 After LINQ:

- Declarative (WHAT you want, not HOW)
- Cleaner & shorter code
- Strongly typed

🔷 2. Why LINQ?

Without LINQ	With LINQ
Verbose loops	One-line queries
Error-prone	Type-safe
Hard to read	SQL-like readable

🔷 3. LINQ Architecture

LINQ works with:

1. **Data Source**
    - Array, List, DB, XML
2. **Query**
    - What you want to retrieve

3. **Execution**
    - Deferred or Immediate

🔷 **4. Basic Example**

```
var numbers = new List<int> { 1, 2, 3, 4, 5, 6 };


// LINQ Query
var result = numbers.Where(n => n > 3);


foreach (var item in result)
{
    Console.WriteLine(item);
}
```

👉 Output:

```
4
5
6
```

🔷 **5. Two Syntax Styles**

✅ **1. Method Syntax (Most Used)**

```
var result = numbers.Where(n => n > 3);
```

✅ **2. Query Syntax (SQL-like)**

```
var result = from n in numbers
             where n > 3
             select n;
```

🔷 6. Important LINQ Concepts

🔹 Deferred Execution

Query runs **only when you iterate**

```
var result = numbers.Where(n => n > 3);

// Not executed yet


foreach (var item in result) // Executed here

{

}
```

🔹 Immediate Execution

Use methods like:

- ToList()
- ToArray()
- Count()

```
var result = numbers.Where(n => n > 3).ToList();
```

🔷 7. LINQ Built-in Methods (With Practical Demos)

🔶 1. Filtering

✅ Where()

```
var employees = new List<int> { 10, 20, 30, 40 };


var result = employees.Where(x => x > 20);
```

```
Console.WriteLine(string.Join(",", result));
```

🔶 **2. Projection**

✅ **Select()**

```
var numbers = new List<int> { 1, 2, 3 };

var squares = numbers.Select(n => n * n);

Console.WriteLine(string.Join(",", squares));
```

🔶 **3. Sorting**

✅ **OrderBy()**

```
var nums = new List<int> { 5, 2, 8, 1 };

var sorted = nums.OrderBy(n => n);

Console.WriteLine(string.Join(",", sorted));
```

✅ **OrderByDescending()**

```
var sorted = nums.OrderByDescending(n => n);
```

🔶 **4. Grouping**

✅ **GroupBy()**

```
var numbers = new List<int> { 1, 2, 3, 4, 5, 6 };

var groups = numbers.GroupBy(n => n % 2);
```

```
foreach (var group in groups)
{
    Console.WriteLine($"Key: {group.Key}");
    foreach (var item in group)
        Console.WriteLine(item);
}
```

🔶 **5. Joining**

✅ **Join()**

```
var students = new[]
{
    new { Id = 1, Name = "John" },
    new { Id = 2, Name = "Sam" }
};

var marks = new[]
{
    new { StudentId = 1, Score = 90 },
    new { StudentId = 2, Score = 80 }
};

var result = students.Join(
    marks,
    s => s.Id,
    m => m.StudentId,
```

```
    (s, m) => new { s.Name, m.Score }
);

foreach (var item in result)
{
    Console.WriteLine($"{item.Name} - {item.Score}");
}
```

🔶 6. Aggregation

✅ Count()

```
var count = numbers.Count();
```

✅ Sum()

```
var sum = numbers.Sum();
```

✅ Average()

```
var avg = numbers.Average();
```

✅ Max() / Min()

```
var max = numbers.Max();
var min = numbers.Min();
```

🔶 7. Element Operators

✅ First()

```
var first = numbers.First();
```

✅ FirstOrDefault()

```
var first = numbers.FirstOrDefault(n => n > 10);
```

✅ **Single()**

```
var single = numbers.Single(n => n == 3);
```

🔶 **8. Partitioning**

✅ **Take()**

```
var top3 = numbers.Take(3);
```

✅ **Skip()**

```
var skip2 = numbers.Skip(2);
```

🔶 **9. Set Operations**

✅ **Distinct()**

```
var nums = new List<int> { 1, 2, 2, 3 };


var distinct = nums.Distinct();
```

✅ **Union()**

```
var result = list1.Union(list2);
```

✅ **Intersect()**

```
var result = list1.Intersect(list2);
```

✅ **Except()**

```
var result = list1.Except(list2);
```

🔶 **10. Quantifiers**

✅ **Any()**

```
var hasEven = numbers.Any(n => n % 2 == 0);
```

✅ **All()**

```
var allPositive = numbers.All(n => n > 0);
```

🔶 **11. Conversion**

✅ **ToList(), ToArray()**

```
var list = numbers.ToList();

var array = numbers.ToArray();
```

🔷 **8. Real-World Example (Important)**

🎯 **Scenario: Employee Filtering**

```
class Employee

{

   public string Name { get; set; }

   public int Salary { get; set; }

   public string Department { get; set; }

}


var employees = new List<Employee>

{

   new Employee { Name = "A", Salary = 50000, Department = "IT" },

   new Employee { Name = "B", Salary = 30000, Department = "HR" },

   new Employee { Name = "C", Salary = 70000, Department = "IT" }

};


// High salary IT employees

var result = employees

   .Where(e => e.Department == "IT")
```

```
    .OrderByDescending(e => e.Salary)
    .Select(e => new { e.Name, e.Salary });

foreach (var emp in result)
{
    Console.WriteLine($"{emp.Name} - {emp.Salary}");
}
```

🔷 9. Method Chaining (Core Strength)

```
var result = numbers
    .Where(n => n > 2)
    .Select(n => n * 10)
    .OrderBy(n => n);
```

🔷 10. Summary

LINQ provides:

✓ Filtering → Where()
✓ Projection → Select()
✓ Sorting → OrderBy()
✓ Grouping → GroupBy()
✓ Aggregation → Sum(), Count()
✓ Joining → Join()
✓ Partition → Take(), Skip()
✓ Set ops → Distinct(), Union()